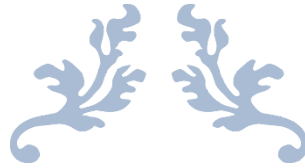


NHẬP MÔN DEVOPS - 23MMT



BÁO CÁO ĐỒ ÁN

Tên đồ án: Project 2 _ CD

MSSV: 23127086

Họ tên: Huỳnh Sĩ Luân

MSSV: 23127148

Họ tên: Ân Tiến Nguyễn An

MSSV: 23127152

Họ tên: Nguyễn Tuấn Anh

MSSV: 23127280

Họ tên: Nguyễn Hiến Tuấn Anh

Họ tên giáo viên vấn đáp:

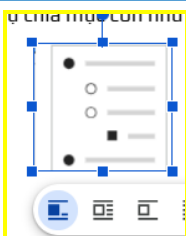
Nguyễn Thanh Quân



Khoa Công nghệ thông tin
Đại học Khoa học tự nhiên TP HCM

MỤC LỤC

1 TỔNG QUAN NHIỆM VỤ.....	3
1.1 Công việc 1.....	4
1.2 Công việc 2.....	4
1.3 CI.....	5
1.4 Tích hợp Docker Hub.....	5
1.5	6
1.6	6
1.7	7
1.8	7
1.9	7
2 CÁC BƯỚC THỰC HIỆN CẤU HÌNH.....	8
2.1 Công cụ 1.....	9
2.2 Công cụ 2.....	9
2.3 Công cụ 3.....	9
2.4 CI.....	10
2.5 Docker Hub.....	10
2.6	13
2.7	13
2.8	13
2.9	14
2.10	14
2.11	14
3 KẾT QUẢ.....	15
3.1	16
3.2	16
3.3	16
3.4	16
3.5 CI.....	17
3.6 Docker hub.....	17
3.7	19
3.8	19
3.9	19



3.10	20
3.11	20

1

TỔNG QUAN NHIỆM VỤ

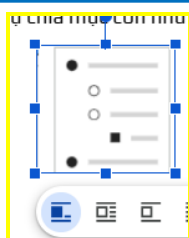
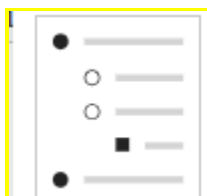


Hướng dẫn

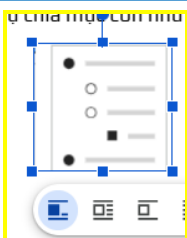
- Tổng quan nhiệm vụ
- Mục tiêu cần đạt
- Điều kiện đầu vào

Yêu cầu chung:

- Font: Play, Size: 11, căn đều, thứ tự chia mục con như mục 1



- Hình ảnh căn giữa, chỉ sử dụng thuộc tính Trong dòng cho ảnh, ghi Figcation (size 11, in nghiêng)



1.1 Công việc 1

Mô tả:

- .
 - .
 -
- .
- .
- .
-

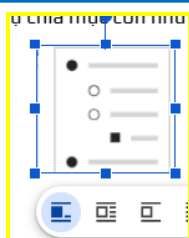
...

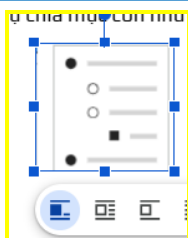
1.2 Công việc 2

Mô tả:

- .
 - .
 -
- .
- .
- .
-

...





1.3 CI

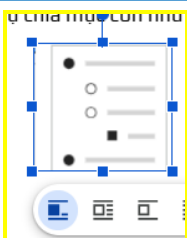
CI có nhiệm vụ bảo đảm khi có thay đổi mã nguồn, workflow tương ứng có thể tự động được kích hoạt để thực hiện các bước kiểm tra cần thiết như build source, test, lint, checkstyle, SonarCloud, OWASP hoặc Trivy tùy theo từng service. Việc này giúp phát hiện sớm lỗi trong quá trình phát triển và bảo đảm chất lượng cơ bản của mã nguồn trước khi chuyển sang bước đóng gói image.

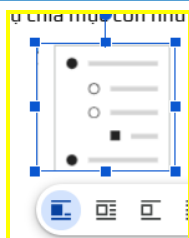
- **Mục tiêu cần đạt** của phần CI là chuẩn hóa lại các workflow service để mọi thay đổi trên source code đều được kiểm tra theo đúng pipeline đã thiết lập. Ngoài ra, workflow cũng phải được tổ chức theo hướng dễ mở rộng để có thể gắn thêm các bước build Docker image phục vụ triển khai.
- **Điều kiện đầu vào** của phần CI bao gồm repository đã có sẵn source code của hệ thống YAS, thư mục **.github/workflows** chứa các workflow service hiện có, cùng với cấu trúc dự án và các Dockerfile tương ứng cho từng service. Đây là cơ sở để tiến hành rà soát, phân loại và cập nhật từng workflow.

1.4 Tích hợp Docker Hub

Tích hợp **Docker Hub** vào pipeline để biến kết quả của CI thành artifact có thể triển khai. Sau khi hoàn thành kiểm tra mã nguồn, workflow cần có khả năng đăng nhập Docker Hub, build image của từng service và push image đó lên registry.

- **Mục tiêu cần đạt** của phần tích hợp Docker Hub là tạo ra hai loại tag image. Với các lần push, image phải được gắn tag theo commit SHA để phục vụ kiểm thử branch phát triển. Với nhánh main, image phải tiếp tục được tạo thêm tag latest để làm phiên bản mặc định phục vụ cho quá trình CD. Kết quả này giúp các bước triển khai tiếp theo có thể xác định đúng image theo từng service và từng phiên bản mã nguồn.
- **Điều kiện đầu vào** của phần này là repository phải được cấu hình đầy đủ hai *GitHub Secrets* gồm **DOCKERHUB_USERNAME** và **DOCKERHUB_TOKEN** để workflow có thể xác thực với Docker Hub. Đồng thời, quy ước đặt tên image cũng cần được thống nhất theo mẫu **docker.io/<DOCKERHUB_USERNAME>/yas-<service>:<tag>** để toàn bộ các service sử dụng cùng một chuẩn đặt tên.

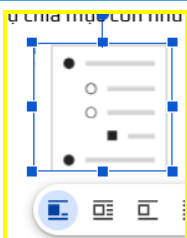




1.5 CD

1.6 Triển khai Jenkins

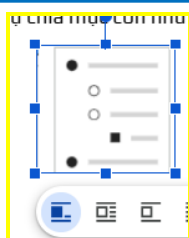
Tích hợp **Jenkins** vào pipeline để



1.7

1.8

1.9



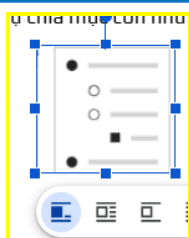
2

CÁC BƯỚC THỰC HIỆN CẤU HÌNH



Hướng dẫn

- Các bước cấu hình/thực hiện
- Hình minh họa theo từng bước
- (Các bước có yêu cầu riêng cần bổ sung theo thấy)



2.1

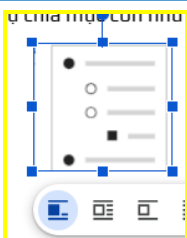
Công cụ 1

2.2

Công cụ 2

2.3

Công cụ 3



2.4 CI

Bước 1. Rà soát và xác định các workflow cần chỉnh sửa

Bước đầu tiên là kiểm tra toàn bộ các workflow hiện có trong repository để xác định những file cần cập nhật. Việc rà soát này giúp bảo đảm chỉ các workflow liên quan trực tiếp đến từng service mới được chỉnh sửa, trong khi các workflow phục vụ mục đích khác như quét bảo mật hoặc kiểm tra chart vẫn được giữ nguyên.

Kết quả rà soát cho thấy có 20 workflow service cần cập nhật, bao gồm các service backend, frontend và BFF. Các workflow như charts-ci.yaml, codeql.yml và gitleaks-check.yaml không thuộc phạm vi build image theo service nên không được chỉnh sửa trong phần việc này.

STT	Tên workflow	Nhóm	Trạng thái	Ghi chú
1	cart-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
2	customer-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
3	inventory-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
4	location-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
5	media-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
6	order-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
7	payment-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
8	payment-paypal-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
9	product-ci.yaml	Backend service	Cần chỉnh sửa	File mẫu tham chiếu chính
10	promotion-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
11	rating-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
12	recommendation-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub

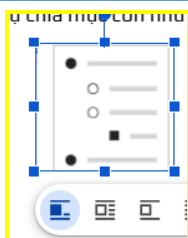
13	sampledata-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
14	search-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
15	tax-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
16	webhook-ci.yaml	Backend service	Cần chỉnh sửa	Chuẩn hóa theo mẫu CI + Docker Hub
17	backoffice-bff-ci.yaml	Frontend/BFF	Cần chỉnh sửa	Chỉnh theo nhóm frontend/BFF
18	storefront-bff-ci.yaml	Frontend/BFF	Cần chỉnh sửa	Chỉnh theo nhóm frontend/BFF
19	storefront-ci.yaml	Frontend	Cần chỉnh sửa	Chỉnh theo nhóm frontend
20	backoffice-ci.yaml	Frontend	Cần chỉnh sửa	Chỉnh cuối vì có thêm Trivy
21	charts-ci.yaml	Charts/Utility	Không chỉnh sửa	Không thuộc phạm vi build image service
22	codeql.yml	Security scan	Không chỉnh sửa	Workflow quét mã bảo mật
23	gitleaks-check.yaml	Secret scan	Không chỉnh sửa	Workflow kiểm tra rò rỉ secret

Danh sách các file trong thư mục .github/workflows

Bước 2. Giữ nguyên và chuẩn hóa các bước kiểm tra chất lượng

Sau khi xác định được các workflow cần chỉnh sửa, nội dung tiếp theo là rà soát cấu trúc của từng workflow để bảo đảm các bước kiểm tra chất lượng hiện có vẫn được giữ nguyên. Tùy theo từng service, workflow vẫn tiếp tục thực hiện các bước như build source, test, lint, checkstyle, SonarCloud, OWASP hoặc Trivy.

Mục tiêu của bước này là bảo đảm phần mở rộng pipeline không làm ảnh hưởng đến chức năng chính của CI. Nói cách khác, trước khi bổ sung phần build image, workflow vẫn phải duy trì đúng vai trò kiểm tra chất lượng mã nguồn cho từng service.

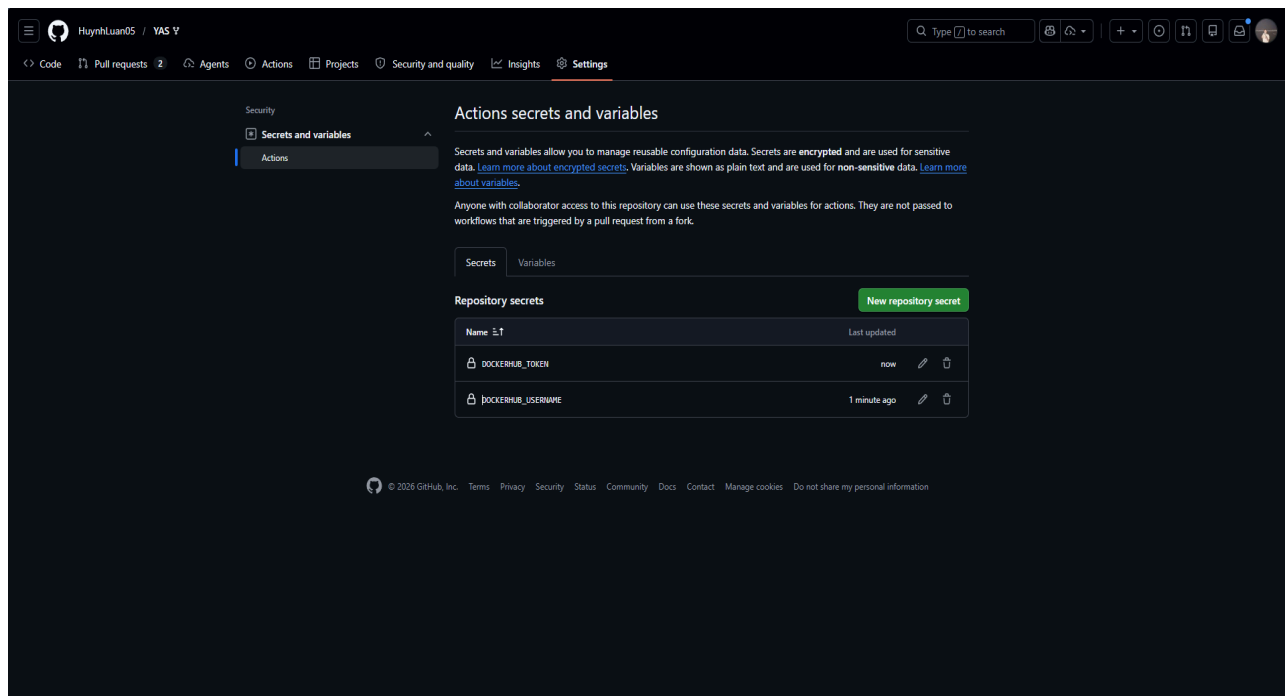


2.5 Docker Hub

Bước 1. Cấu hình thông tin xác thực Docker Hub

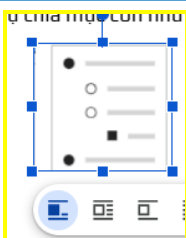
Sau khi hoàn thiện phần rà soát CI, bước tiếp theo là cấu hình cơ chế xác thực với Docker Hub. Để bảo đảm an toàn thông tin, repository được bổ sung hai *GitHub Secrets* là **DOCKERHUB_USERNAME** và **DOCKERHUB_TOKEN**. Hai biến này được sử dụng trong các workflow để đăng nhập Docker Hub trước khi thực hiện build và push image.

Việc sử dụng **GitHub Secrets** giúp tránh lưu trực tiếp tài khoản và mật khẩu trong file workflow, đồng thời cho phép toàn bộ các service sử dụng thống nhất một cơ chế xác thực chung.



Cấu hình GitHub Secrets gồm DOCKERHUB_USERNAME và DOCKERHUB_TOKEN

Bước 2. Bổ sung bước build và push image theo commit SHA



Sau khi cấu hình secrets, các workflow được cập nhật để bổ sung bước **đăng nhập Docker Hub** bằng **docker/login-action@v3** và bước **build, push image** bằng **docker/build-push-action@v6**. Với các sự kiện push, image của service tương ứng được build và gắn tag theo **`${{ github.sha }}`**.

Việc gắn tag theo **commit SHA** giúp mỗi lần commit mới sinh ra một image riêng biệt, từ đó dễ dàng truy vết giữa mã nguồn và artifact được tạo ra. Đây cũng là yêu cầu quan trọng của đề bài đối với phần CI dành cho branch phát triển.

Quy ước đặt tên image được sử dụng theo mẫu:

```
docker.io/<DOCKERHUB_USERNAME>/yas-<service>:<tag>
```

Ví dụ:

```
docker.io/<username>/yas-cart:<commit-sha>
```

```
docker.io/<username>/yas-product:<commit-sha>
```

```
docker.io/<username>/yas-tax:<commit-sha>
```

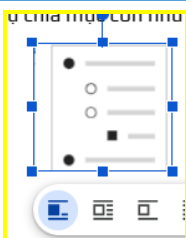
Bước 3. Duy trì image mặc định latest trên nhánh main

Bên cạnh việc tạo image theo **commit SHA**, workflow còn được bổ sung bước build và push image với tag latest khi mã nguồn được đẩy lên nhánh main. Image latest đóng vai trò là phiên bản mặc định, được dùng trong những trường hợp không cần chỉ định rõ commit cụ thể hoặc khi các service khác trong hệ thống sử dụng phiên bản ổn định gần nhất.

Cơ chế này giúp pipeline hỗ trợ đồng thời hai nhu cầu: kiểm thử branch bằng **image theo SHA** và triển khai mặc định bằng **image latest**.

Bước 4. Kiểm tra kết quả push image trên GitHub Actions và Docker Hub

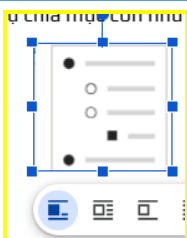
Sau khi hoàn tất cập nhật workflow, pipeline được chạy thử để kiểm tra việc đăng nhập Docker Hub, build image và push artifact thành công. Kết quả trên GitHub Actions giúp xác nhận workflow hoạt động đúng logic, trong khi danh sách tag trên Docker Hub là bằng chứng trực tiếp cho thấy image đã được phát hành thành công.



2.6 CD

2.7

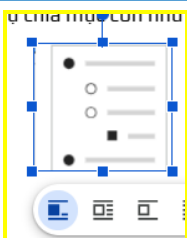
2.8



2.9

2.10

2.11



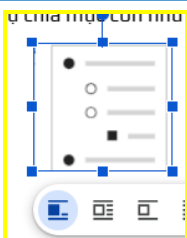
3

KẾT QUẢ



Hướng dẫn

- Kết quả đạt được (Hình ảnh minh họa kèm theo)
- Khó khăn / lưu ý nếu có

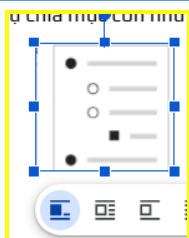


3.1

3.2

3.3

3.4



3.5 CI

Sau khi hoàn thiện phần CI, các workflow của các service trong hệ thống YAS đã được rà soát và chuẩn hóa lại theo cùng một hướng cấu hình. Mỗi workflow có thể tự động được kích hoạt khi source code của service tương ứng thay đổi, từ đó thực hiện các bước kiểm tra cần thiết như build source, test, lint, checkstyle, SonarCloud, OWASP hoặc Trivy tùy theo từng loại service.

Kết quả quan trọng nhất của phần này là pipeline CI vẫn giữ được đầy đủ vai trò kiểm tra chất lượng mã nguồn trước khi chuyển sang bước đóng gói image. Điều này giúp phát hiện lỗi sớm trong quá trình phát triển, đồng thời tạo ra một luồng xử lý thống nhất cho các service backend, frontend và BFF trong hệ thống.

Các kết quả chính **đạt được** của phần CI bao gồm:

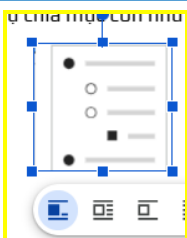
- Rà soát và xác định đầy đủ các workflow service cần chỉnh sửa.
- Chuẩn hóa lại các workflow theo cùng một mẫu xử lý.
- Giữ nguyên các bước kiểm tra chất lượng hiện có trong workflow.
- Bảo đảm workflow vẫn hoạt động đúng sau khi được mở rộng thêm chức năng build image.

Khó khăn lớn nhất trong phần CI là số lượng workflow service cần xử lý tương đối nhiều, trong khi cấu trúc giữa các workflow không hoàn toàn giống nhau. Một số workflow thuộc backend Java, một số thuộc frontend hoặc BFF, vì vậy cần rà soát kỹ để tránh chỉnh sai context build hoặc làm ảnh hưởng đến các bước kiểm tra chất lượng sẵn có.

Một lưu ý quan trọng là khi mở rộng workflow, cần giữ nguyên các bước kiểm tra chất lượng ban đầu. Việc bổ sung bước build image không được làm mất hoặc làm sai lệch các bước như build source, test, lint, checkstyle, SonarCloud, OWASP hoặc Trivy. Điều này giúp phần mở rộng CI vẫn phù hợp với mục tiêu ban đầu của pipeline.

3.6 Docker hub

Sau khi hoàn thiện phần tích hợp Docker Hub, các workflow đã có thể đăng nhập Docker Hub thành công, build image đúng theo từng service và push image lên registry theo đúng quy ước đặt tên đã thống nhất. Với các lần push, image được gắn tag theo commit SHA để phục vụ kiểm thử branch phát triển. Với nhánh main, workflow tạo thêm image latest để làm phiên bản mặc định phục vụ triển khai.



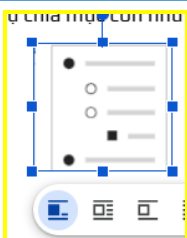
Kết quả này cho thấy pipeline không chỉ dừng ở mức kiểm tra mã nguồn mà đã tạo ra artifact triển khai chuẩn hóa là Docker image. Đây là đầu vào quan trọng cho bước CD, giúp các công cụ triển khai như Jenkins hoặc Kubernetes có thể pull đúng image theo từng service và từng phiên bản cụ thể.

Các kết quả chính **đạt được** của phần tích hợp Docker Hub bao gồm:

- Cấu hình thành công GitHub Secrets để xác thực với Docker Hub.
- Bổ sung bước đăng nhập Docker Hub trong các workflow service.
- Build và push image thành công với tag theo commit SHA.
- Duy trì image latest trên nhánh main làm phiên bản mặc định.
- Tạo artifact chuẩn hóa phục vụ cho bước CD và triển khai hệ thống.

Khó khăn chính của phần tích hợp Docker Hub là phải bảo đảm tất cả các workflow đều sử dụng đúng thông tin xác thực, đúng quy ước đặt tên image và đúng context build của từng service. Nếu chỉ một workflow bị cấu hình sai tag hoặc sai đường dẫn context thì quá trình build và push image sẽ thất bại hoặc tạo ra image không đúng mong muốn.

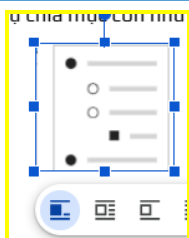
Ngoài ra, cần lưu ý rằng tên của **GitHub Secrets** phải chính xác và được khai báo thống nhất trong toàn bộ workflow. Nếu sai tên secret hoặc thiếu quyền truy cập, bước đăng nhập Docker Hub sẽ thất bại và làm dừng toàn bộ quá trình push image. Một lưu ý khác là phải tách rõ hai luồng xử lý giữa image theo **commit SHA** và **image latest**, tránh trường hợp ghi đè sai tag hoặc làm mất khả năng truy vết giữa image và mã nguồn.



3.7

3.8

3.9



3.10

3.11

